

If there is no initial SQL, the database will be empty. If there is no `pgdatadir` given, Docker will keep the data in an automatic volume. `pg.env` is where you plan to keep all the environment variables. You can put them inside the Compose file itself, but that's a bad idea (for example, someone might ask you to share your Compose file). Yes, all the file names are of your own choice. Map them using the Compose file and only the targets matter.

Now let's define the contents of `docker-compose.yml`, the Compose file:

```
version: "3.3"
services:
  db:
    image: postgres
    container_name: mypostgres2
    volumes:
      - ./initdb.d:/docker-entrypoint-initdb.d/
      - ./pgdatadir:/var/lib/postgresql
    env_file:
      - pg.env
    ports:
      - "9000:5432"
```

It should already make sense to you. You are defining just one service (`db`, again a name of your choice) here. The lines under `volumes`: map the local directories to the container's file system. Similarly, there's a mapping going on under `ports`. Here you are mapping the container's internal port 5432 (the default Postgres port) to the host machine's port 9000 (once again, your choice). This mapping is not needed if the database is only accessed from the services defined in the same Compose file. But with this mapping, the service is exposed as part of the host itself, making it available to any application run in the host, LAN, or even the Internet, as long as the routers and firewalls permit. Now you know one security pitfall. I recommend you do some research on exposing ports and container IPs (also, try the command `docker inspect CONTAINER_NAME`).

Okay, let's see the contents of the environment file:

```
POSTGRES_PASSWORD=mypgpas
```

The `env` file has a syntax that expects one variable per line. Anything put after the equals symbol is part of the value, which means no quotes or escaping.

Finally, how do you start the service? Make sure you are inside the directory `myproject` and run this:

```
docker-compose up -d
```

You can still use the basic Docker commands like `docker exec` and `docker logs`. However, to stop the services, it's better to run:

```
docker-compose down
```

Connecting to the database

To interact with the database, you can use `docker exec`. But now that the port is open, you can connect from a client that is outside Docker (say `psql` installed system-wide using `apt install`), or from your application, just like you connect always. The host will be `localhost` and the port will be 9000, based on the above example. If your application is run by the same Compose file, you can use the container's name (here, `mypostgres2`) as the host.

MySQL

Things that change with MySQL include the environment variables, the path of the data directory and the internal port.

The useful variables are:

- `MYSQL_DATABASE`
- `MYSQL_USER`
- `MYSQL_PASSWORD`

The data directory is `/var/lib/mysql` and the default MySQL port is 3306.

Monitoring and clean up

You can use the command `docker ps` to list the running containers. `docker stop CONTAINER_NAME` is how you stop a container, but it's recommended to stop it using `docker-compose down` if you have started it using `docker-compose up`. Run `docker system prune` to clean up unused files and cache. Docker volumes can also consume disk space. Run the command `docker volume` to learn how you can manage them.

If you want to stop the whole Docker service for some reason, run `systemctl stop docker`. You can use `systemctl` to restart Docker also. 

References

- [1] https://hub.docker.com/_/postgres/
- [2] https://hub.docker.com/_/mysql/
- [3] <https://docs.docker.com/compose/>

By: Nandakumar Edamana

The author is a developer and technology writer who primarily focuses on software freedom, privacy and other ethical issues. He has authored a couple of books and several articles for mainstream publishers. A GNU/Linux user for more than ten years, he releases his software products under free software licences.